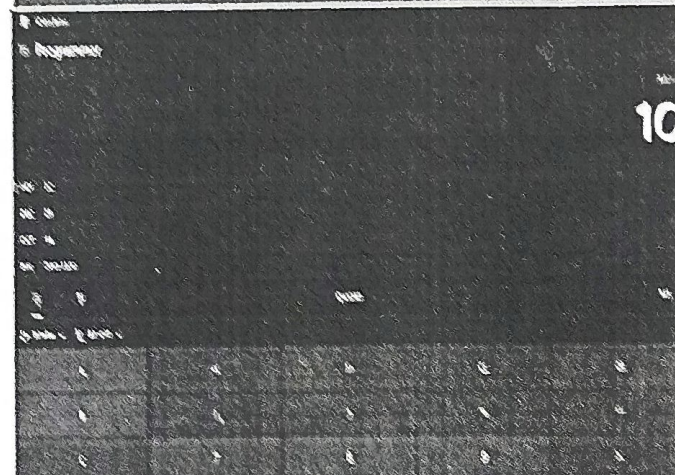
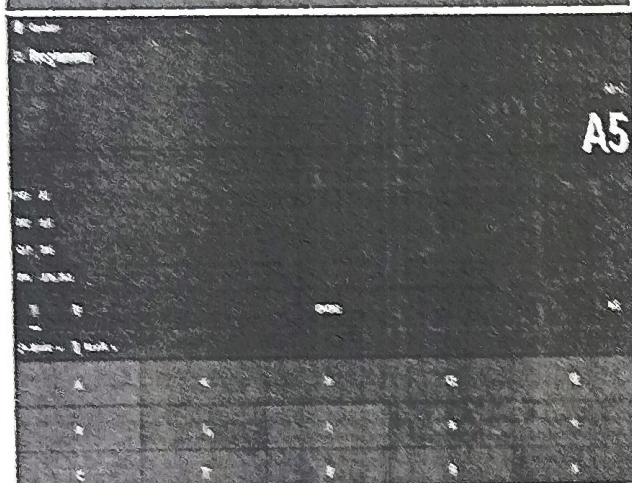
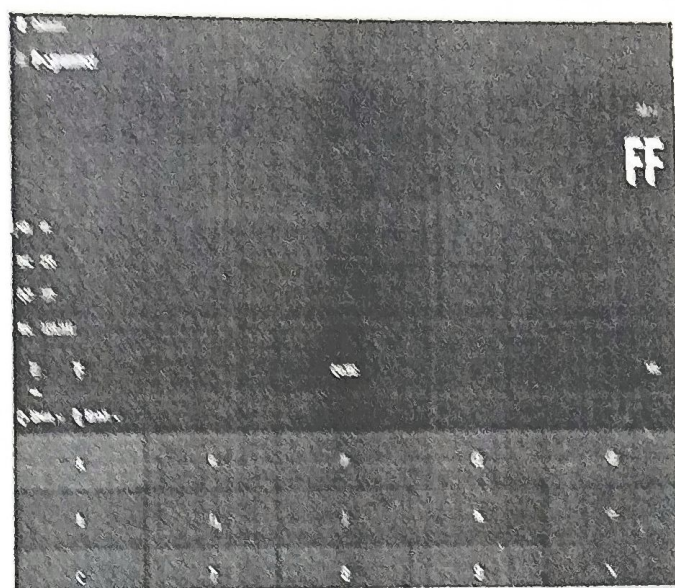
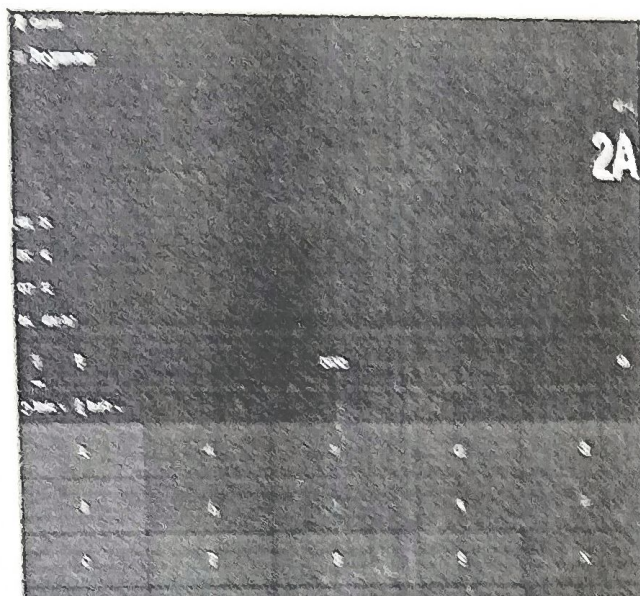


Q1. A software developer is verifying memory values before debugging a program. To validate the stored values, use the Windows Calculator (Programmer Mode) to convert the given hexadecimal values into their binary and decimal equivalents. Analyse the converted values to determine how hexadecimal simplifies binary representation and identify how an incorrect hexadecimal value could affect memory interpretation during program execution. Support your analysis with suitable conversion results.



Example Conversion Table.

Hex	Binary	Decimal
2A	00101010	42
FF	11111111	255
1C	00011100	28
A5	10100101	165

ANALYSIS

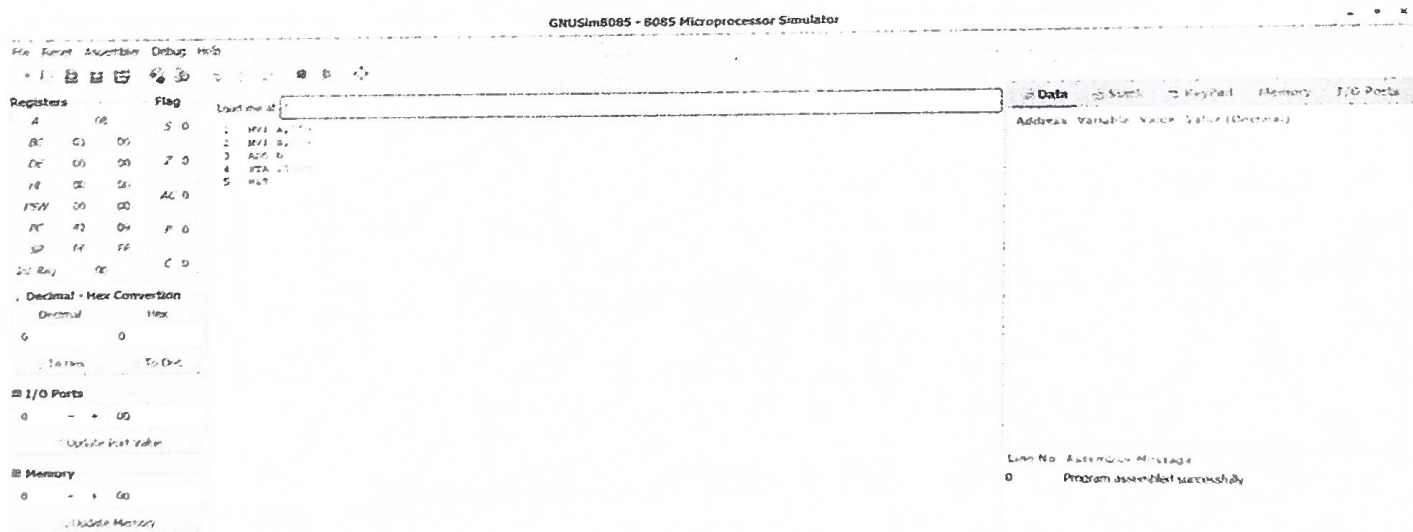
- Hexadecimal represent every 4 binary bits using one digit
- It makes long binary number easier to read.
- It reduces human error while debugging memory.

* If an incorrect hexadecimal value is entered, the binary value change completely, leading to incorrect memory interpretation, wrong calculation or program crashes.

Conclusion

Window calculator successfully converted hexadecimal value into binary and decimal. Hexadecimal simplifies binary representation and improves memory debugging.

Q 2. A programmer wants to examine how different instructions are executed by an 8085 microprocessor. To observe the instruction execution process, write and execute an 8085 program containing data transfer and arithmetic instructions using an 8085 simulator. Observe the changes in the Program Counter (PC), Accumulator (A), and Flag Register after each instruction. Analyse how the processor performs the instruction fetch, decode, and execute stages for each instruction and explain how these stages contribute to the final output.



OBSERVATION TABLE

Instruction	PC	A	Zero	Sign	Carry
MVI A, 05H	Next address	05H	0	0	0
MVI B, 03H	Next Address	05H	0	0	0
ADD B	Next Address	08H	0	0	0
STA 2500H	Next Address	08H	0	0	0
HLT	STOP	08H	0	0	0

ANALYSIS.

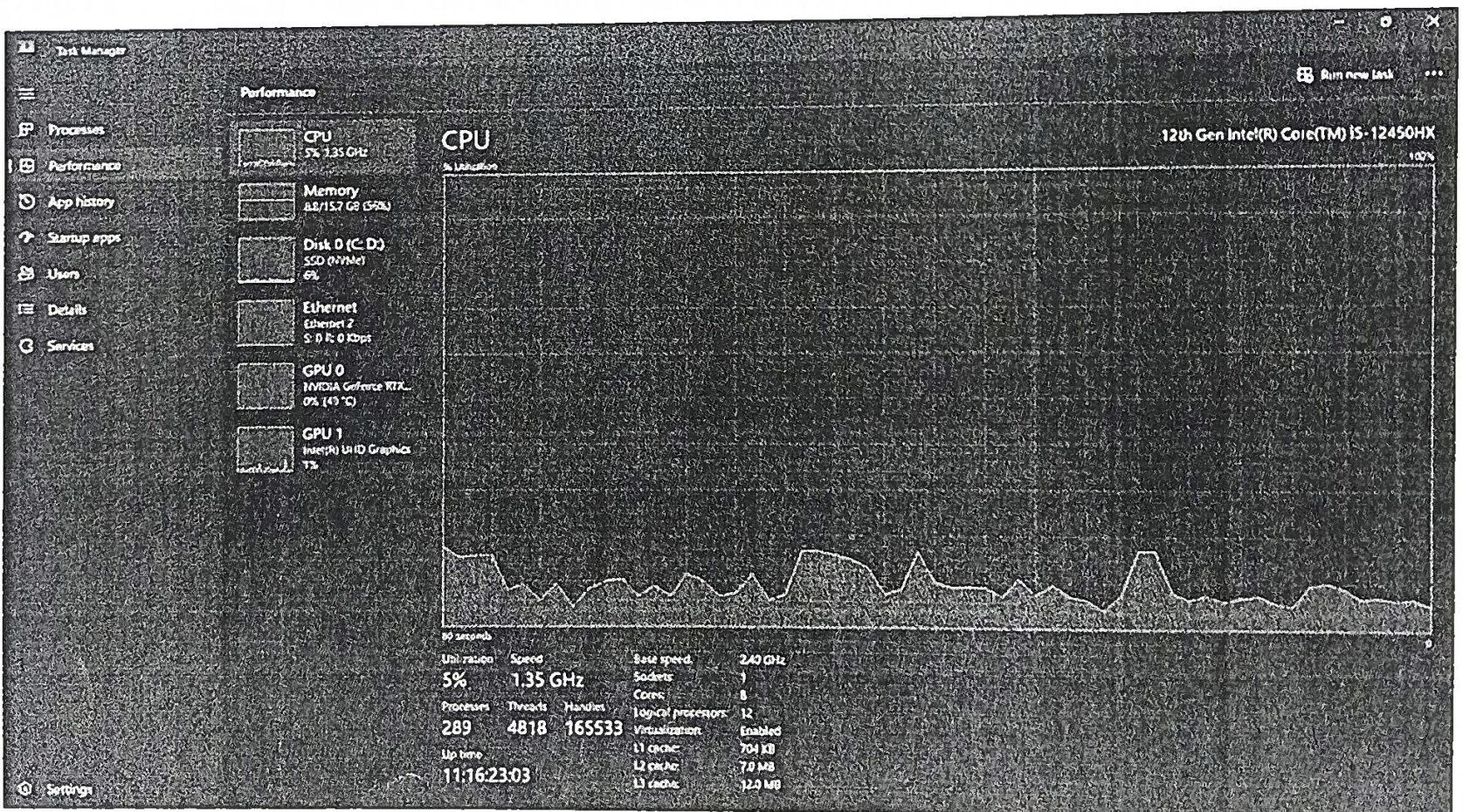
- * Fetch - CPU fetches instruction from memory
- * Decode - CPU identifies the operation (move, add, store).
- * Execute - CPU perform the instruction and updates registers.

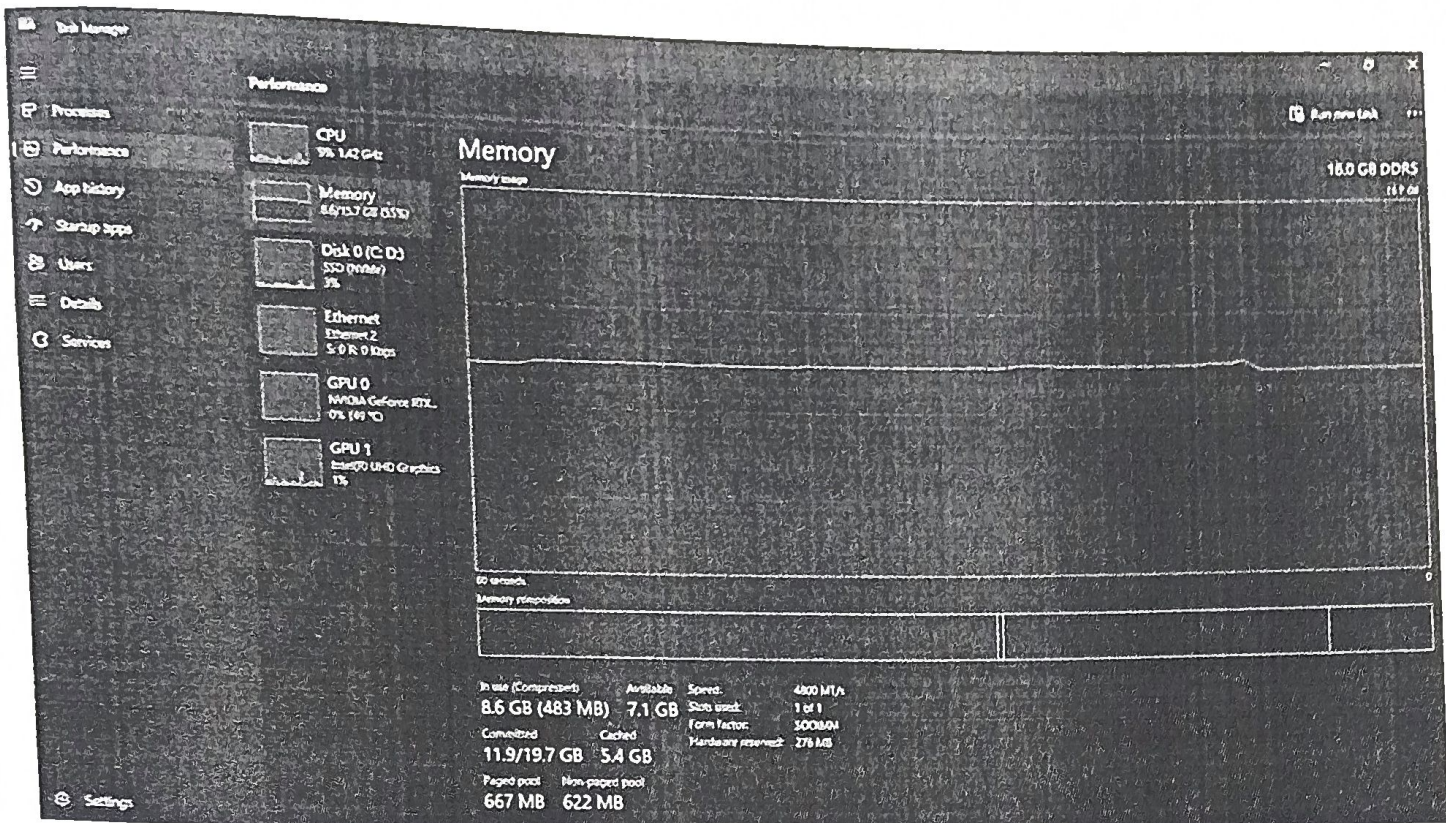
The Program counter increase after every instruction.
 The accumulator stores the arithmetic result.
 The flag registers updates based on the result.

CONCLUSION

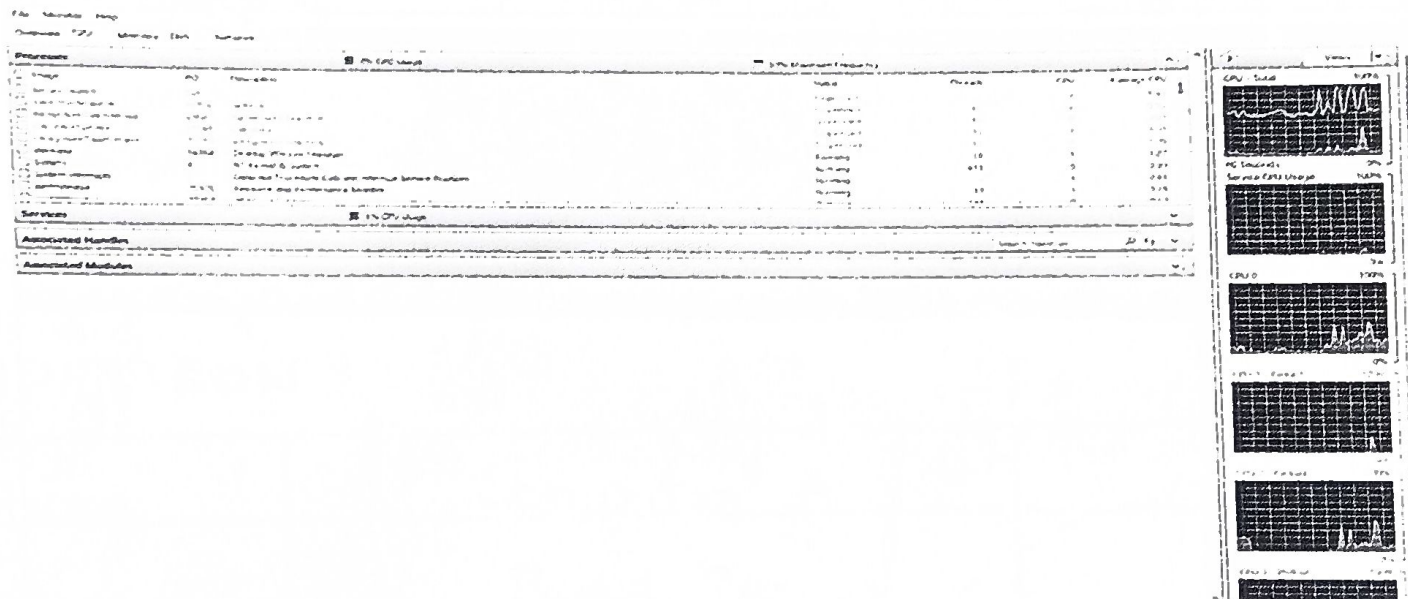
The simulator shows how the processor perform
 Fetch → Decode → Execute for every instruction
 resulting in the final ~~of~~ output stored in memory

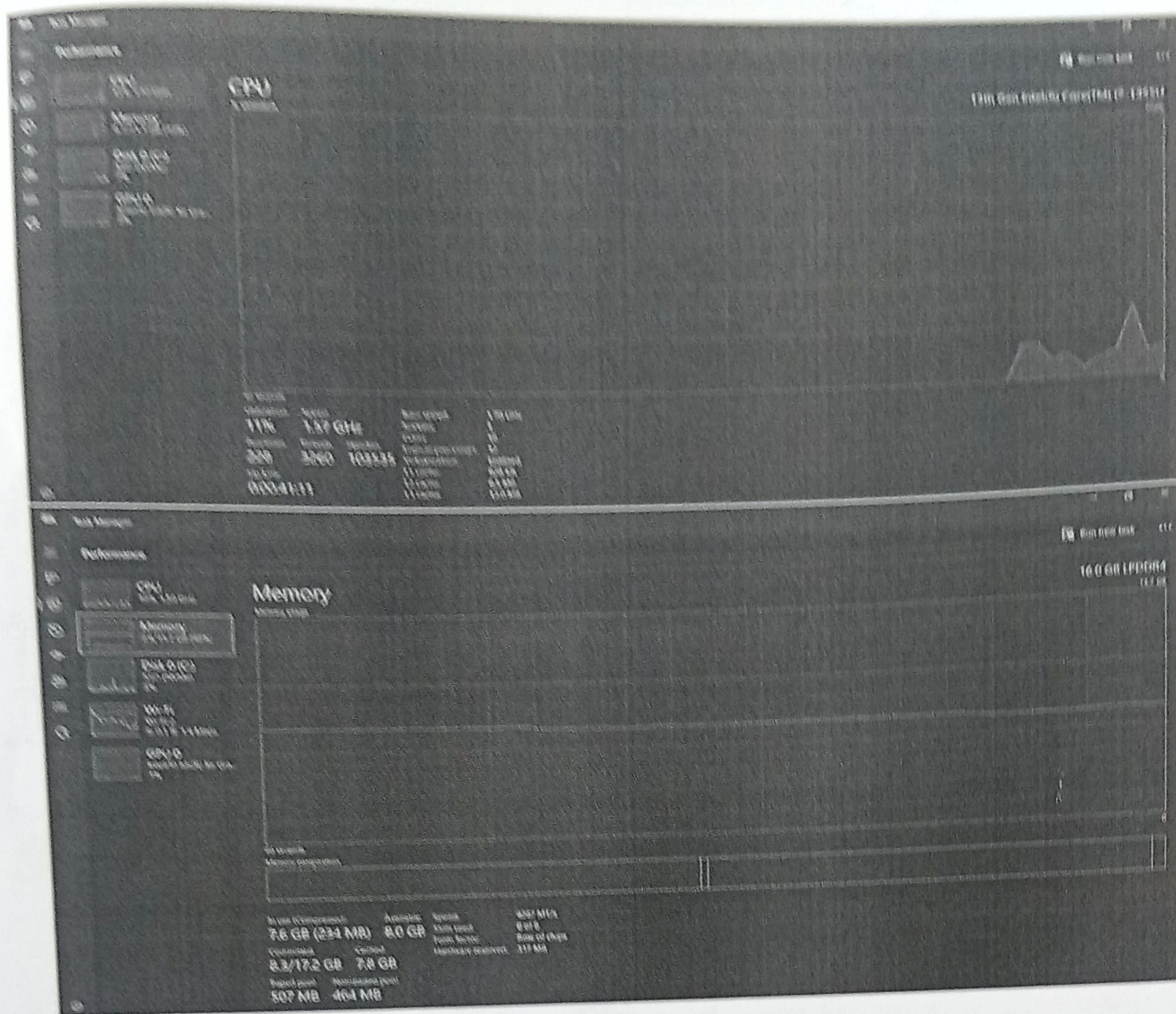
COMPUTER 1





COMPUTER 2





COMPARISON

Feature	Computer A	Computer B
Processor	Intel i5	Intel i7
Base speed	2.40 GHz 2.40 GHz	1.70 GHz. 1.70 GHz
Current speed	1.35 GHz	1.57 GHz
Cores	8	10
Logical Processors	12	12
RAM	16 GB	16 GB
Cache	5 MB	12 MB
System Type	64-bit	64-bit

- * Higher Processor Speed
- * More Cores
- * More logical processors
- * Larger Cache
- * More RAM

Therefore, Computer B can execute application faster, support multitasking more efficiently and provide better overall performance.

CONCLUSION

Computer B is expected to perform better because of its stronger processor, higher clock speed, large cache and greater memory capacity.